

Guía de apuntes: Programación Móvil con Flutter y Dart

Resumen y explicación detallada del código del repositorio [angelloaroni/ProgramacionMovilEjercicios](#) (rama `main`). Los Ejercicios 1, 2 y 3 están explicados completos, línea por línea. `EjercicioWidgets.dart` y `Resumen.dart` se resumen con sus ideas clave.

Contenido del repositorio

Archivo	Tamaño (según GitHub)	Tema principal	Nivel de detalle aquí
<code>Ejercicio1.dart</code>	49 líneas	Row con Expanded y flex (proporción 4 : 1)	Completo
<code>Ejercicio2.dart</code>	79 líneas	Column + Row anidados: 4 bandas de colores	Completo
<code>Ejercicio3.dart</code>	222 líneas	Pantalla tipo "artículo" (estilo Material 3) con métodos auxiliares	Completo
<code>EjercicioWidgets.dart</code>	120 líneas	StatefulWidget, setState, rotación de colores y flex	Resumen
<code>Resumen.dart</code>	437 líneas	InheritedWidget, pestañas (TabBar), GestureDetector, ListView	Resumen

Todos los archivos son aplicaciones Flutter de un solo archivo: cada uno tiene su propio `main()`.

Índice

- 0. Cómo ejecutar cualquier ejercicio
- 1. Conceptos base (Dart, widgets, layout)
- 2. Ejercicio 1: Row con proporciones
- 3. Ejercicio 2: bandas con Column y Row anidados
- 4. Ejercicio 3: pantalla tipo artículo
- 5. `EjercicioWidgets.dart`: estado y `setState`
- 6. `Resumen.dart`: `InheritedWidget`, pestañas y más
- 7. Chuleta final, errores típicos y autoevaluación

0. Cómo ejecutar cualquier ejercicio

Como cada archivo tiene su propio `main()`, son independientes entre sí.

Opción A: proyecto Flutter local

```
flutter create mi_app
cd mi_app
# Reemplaza el contenido de lib/main.dart por el código del ejercicio
flutter run
```

Opción B: ejecutar un archivo concreto sin reemplazar `main.dart`

```
flutter run -t ruta/al/Ejercicio1.dart
```

(`-t` es la abreviatura de `--target`: indica cuál archivo contiene el `main()` a ejecutar.)

Opción C: DartPad ([dartpad.dev](#)), que permite pegar código Flutter y verlo en el navegador sin instalar nada.

1. Conceptos base

1.1 Dart: lo mínimo que aparece en el código

Sintaxis	Qué significa	Dónde aparece
<code>import</code> <code>'package:flutter/material.dart';</code>	Importa la librería Material de Flutter (widgets + diseño Material). Es la primera línea de todos los archivos.	Todos
<code>void main() { ... }</code>	Punto de entrada del programa. Dart empieza a ejecutar aquí.	Todos
<code>final</code>	Variable que se asigna una sola vez (en tiempo de ejecución). La referencia no cambia, pero el contenido de una lista <code>final</code> sí puede modificarse.	<code>final List<Color></code> <code>_colors</code>
<code>const</code>	Constante en tiempo de compilación . Flutter puede reutilizar la misma instancia y evitar trabajo innecesario. Un <code>const Row(...)</code> exige que todo lo de adentro también sea constante.	<code>const</code> <code>SizeBox(width: 20)</code>
Parámetros nombrados <code>{ ... }</code>	Se pasan como <code>nombre: valor</code> , en cualquier orden.	<code>Padding(padding: ... , child: ...)</code>
<code>required</code>	Marca un parámetro nombrado como obligatorio.	<code>required this.color</code>
<code>super.key</code>	Pasa el parámetro <code>key</code> al constructor de la clase padre (<code>StatelessWidget</code> , etc.). Forma corta moderna de escribir el constructor de un widget.	<code>const</code> <code>MyApp({super.key})</code>
<code>?, ?., ??</code>	Tipo? puede ser null; <code>a?.b</code> accede solo si <code>a</code> no es null; <code>a ?? b</code> usa <code>b</code> si <code>a</code> es null.	<code>config?.appTitle ??</code> <code>'Widgets Demo'</code>
<code>_nombre</code>	El guion bajo hace el elemento privado al archivo .	<code>_buildThumbnail,</code> <code>_ColoredCard</code>
<code>@override</code>	Indica que se está reemplazando un método de la clase padre.	<code>build(...)</code>
<code>'\${expresión}'</code>	Interpolación de texto: inserta el valor de una expresión dentro de una cadena.	<code>'Flex:</code> <code>\$_proportions[0]]'</code>
Cadenas adyacentes	Dos literales consecutivos se concatenan en uno solo (sirve para partir textos largos).	Los textos "Lorem ipsum"
<code>static</code>	Método que pertenece a la clase, no a una instancia.	<code>static Widget</code> <code>_buildThumbnail(...)</code>
Genéricos <code><T></code>	Indican el tipo de contenido.	<code>List<Color> ,</code> <code>State<MiWidget></code>
Coma final (<i>trailing comma</i>)	Coma después del último argumento. Hace que el formateador (<code>dart format</code>) ponga cada argumento en su línea; por eso el código de Flutter tiene tantas comas al final.	Todo el código

1.2 Todo es un widget y forman un árbol

En Flutter la interfaz se construye componiendo **widgets** (descripciones inmutables de una parte de la pantalla). Unos widgets contienen a otros y así forman un **árbol de widgets**:

```
MaterialApp
├── Scaffold
│   ├── Column
│   │   ├── Text
│   │   └── Row
│   │       ├── Icon
│   │       └── Text
```

Regla práctica para leer código Flutter: **de afuera hacia adentro**. El widget más externo es el "contenedor" y cada `child:` / `children:` es lo que hay dentro.

- `child:` recibe **un** widget.
- `children:` recibe una **lista** de widgets (`[ ... ]`).

1.3 StatelessWidget VS StatefulWidget

	<code>StatelessWidget</code>	<code>StatefulWidget</code>
Estado interno que cambia	No	Sí
Se redibuja cuando...	El padre lo reconstruye	Además, cuando se llama a <code>setState</code>

Estructura	Una clase con <code>build</code>	Dos clases: el widget y su <code>State</code>
Ejemplos en el repo	<code>MyApp</code> (Ejercicio 3), <code>_ColoredCard</code>	<code>EjercicioRotacionPantalla</code> , <code>RotatingFlexExercise</code>

Los Ejercicios 1 y 2 ni siquiera definen una clase: arman el árbol directamente dentro de `runApp(...)`.

1.4 El método `build` y `BuildContext`

Widget `build(BuildContext context)` es el método que **devuelve el árbol de widgets** que representa a ese widget en pantalla. El `BuildContext` indica en qué lugar del árbol está el widget; sirve, por ejemplo, para preguntarle a los ancestros por datos (como hace `AppConfigInherited.of(context)` en `Resumen.dart`).

1.5 El modelo de layout (la regla de oro)

Las restricciones bajan, los tamaños suben, y el padre decide la posición.

1. El padre le pasa al hijo unas **restricciones** (ancho y alto mínimo/máximo).
2. El hijo elige un tamaño dentro de esas restricciones y se lo comunica al padre.
3. El padre coloca al hijo.
 - **Restricción estricta (*tight*)**: mínimo = máximo. El hijo no tiene opción (ej. `SizeBox(height: 200)` impone alto 200).
 - **Restricción holgada (*loose*)**: mínimo 0 hasta un máximo. El hijo puede ser tan pequeño como quiera (ej. lo que `Center` le da a su hijo).

Ejes en `Row` y `Column`:

	<code>Row</code>	<code>Column</code>
Eje principal (<i>main axis</i>)	Horizontal	Vertical
Eje cruzado (<i>cross axis</i>)	Vertical	Horizontal

- `mainAxisAlignment`: cómo se reparten los hijos a lo largo del eje principal (por defecto `start`).
- `crossAxisAlignment`: cómo se alinean en el eje cruzado (por defecto `center`).
- `mainAxisSize`: `max` (por defecto, ocupa todo el eje principal) o `min` (solo lo necesario).

Cómo reparte el espacio un `Row` / `Column` (algoritmo simplificado):

1. Mide los hijos que **no** son flexibles (los `SizeBox`, `Text`, `Icon`...) y suma su tamaño.
2. `espacio libre = espacio total - suma de los tamaños fijos`.
3. Suma los `flex` de todos los `Expanded`.
4. Cada `Expanded` recibe `espacio libre × (su flex / suma de flex)`.

`Expanded` es equivalente a `Flexible(fit: FlexFit.tight)`: obliga al hijo a ocupar **todo** el espacio que le toca.

1.6 Widgets que aparecen en los ejercicios

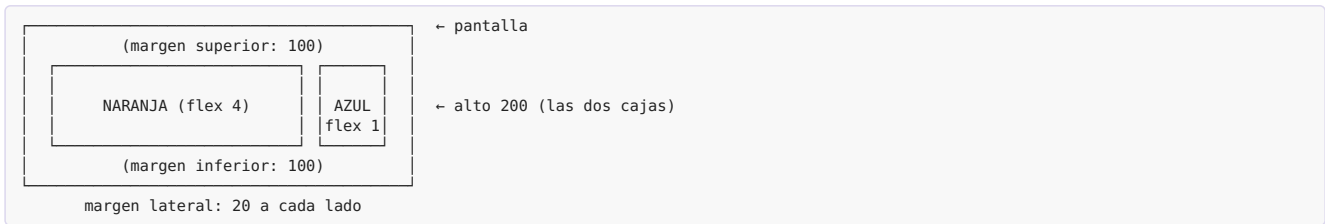
Widget	Para qué sirve	Propiedades clave
<code>MaterialApp</code>	Raíz de una app con diseño Material (tema, navegación, pantalla inicial).	<code>home</code> , <code>debugShowCheckedModeBanner</code> , <code>title</code>
<code>Scaffold</code>	Estructura básica de una pantalla (barra superior, cuerpo, botón flotante...).	<code>body</code> , <code>backgroundColor</code> , <code>appBar</code>
<code>Directionality</code>	Define la dirección del texto (izquierda→derecha o derecha→izquierda).	<code>textDirection</code> , <code>child</code>
<code>Padding</code>	Añade espacio interior alrededor de su hijo.	<code>padding</code> , <code>child</code>
<code>EdgeInsets</code>	Valores de márgenes/relleno.	<code>.all()</code> , <code>.symmetric(horizontal, vertical)</code> , <code>.only()</code> , <code>.fromLTRB()</code>
<code>Center</code>	Centra a su hijo.	<code>child</code>
<code>SizeBox</code>	Caja de tamaño fijo. Con <code>child</code> fuerza su tamaño; sin <code>child</code> sirve de espaciador .	<code>width</code> , <code>height</code> , <code>child</code>
<code>Row</code> / <code>Column</code>	Colocan hijos en horizontal / vertical.	<code>children</code> , <code>mainAxisAlignment</code> , <code>crossAxisAlignment</code>
<code>Expanded</code>	Hace que un hijo de <code>Row</code> / <code>Column</code> ocupe el espacio sobrante según su <code>flex</code> .	<code>flex</code> , <code>child</code>

Container	Caja versátil: color, tamaño, padding, bordes, decoración.	color, decoration, width, height, padding, child
BoxDecoration	Decoración de una caja (color, bordes redondeados, sombra...).	color, borderRadius
BorderRadius.circular(r)	Esquinas redondeadas con radio r.	
Text / TextStyle	Texto y su estilo.	fontSize, fontWeight, color, height, maxLines, overflow
Icon	Muestra un ícono de la fuente Material Icons.	Icons.xxx, size, color
SingleChildScrollView	Permite hacer scroll a un contenido más alto que la pantalla.	child
Spacer	Espacio flexible que empuja a los demás widgets (es un Expanded vacío).	flex

2. Ejercicio 1: Row con proporciones

Objetivo: dibujar dos rectángulos redondeados en una fila, uno naranja y otro azul, con proporción de ancho **4 : 1** y una separación fija de 20 entre ambos.

Resultado visual (esquema):



2.1 Código completo

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    MaterialApp(
      home: Scaffold(
        body: Directionality(
          textDirection: TextDirection.ltr,
          child: Padding(
            padding: const EdgeInsets.symmetric(
              horizontal: 20,
              vertical: 100,
            ),
            child: Center(
              child: SizedBox(
                height: 200,
                child: Row(
                  children: [
                    Expanded(
                      flex: 4,
                      child: Container(
                        decoration: BoxDecoration(
                          color: Colors.orange,
                          borderRadius: BorderRadius.circular(20),
                        ),
                      ),
                    ),
                    const SizedBox(width: 20),
                    Expanded(
                      flex: 1,
                      child: Container(
                        decoration: BoxDecoration(
                          color: Colors.blue,
                          borderRadius: BorderRadius.circular(20),
                        ),
                      ),
                    ),
                  ],
                ),
              ),
            ),
          ),
        ),
      ),
    ),
  );
}
```

2.2 Árbol de widgets

```
MaterialApp
├── Scaffold
│   ├── Directionality (ltr)
│   │   ├── Padding (horizontal 20, vertical 100)
│   │   │   ├── Center
│   │   │   │   ├── SizedBox (alto 200)
│   │   │   │   │   ├── Row
│   │   │   │   │   │   ├── Expanded (flex 4) → Container naranja, radio 20
│   │   │   │   │   │   ├── SizedBox (ancho 20)
│   │   │   │   │   │   └── Expanded (flex 1) → Container azul, radio 20
```

2.3 Explicación paso a paso (de afuera hacia adentro)

1. `import 'package:flutter/material.dart';` Trae todos los widgets de Flutter con estilo Material (`MaterialApp`, `Scaffold`, `Row`, `Container`, `Colors`...). Sin esta línea nada de lo demás compila.

2. `void main() { runApp(...); }` `main` es el punto de entrada. `runApp` recibe un widget, lo convierte en la **raíz** del árbol y lo muestra en pantalla. Aquí se le pasa directamente un `MaterialApp` (no hay clases propias).
3. `MaterialApp(home: ...)` Es la raíz de una app con diseño Material. Provee tema, sistema de navegación y otros servicios. `home` es la **primera pantalla** que se muestra. Como no se especifican `theme` ni `title`, usa los valores por defecto (tema claro; en versiones recientes de Flutter, Material 3 por defecto).
4. `Scaffold(body: ...)` Es el "esqueleto" de una pantalla Material: puede tener `AppBar`, `body`, `floatingActionButton`, `drawer`, etc. Aquí solo se usa `body`, que es el contenido principal. También aporta el color de fondo de la pantalla (blanco por defecto).
5. `Directionality(textDirection: TextDirection.ltr, ...)` Le dice a los widgets hijos que el sentido de lectura es izquierda→derecha (*left-to-right*). `Row` necesita saberlo para decidir qué hijo va a la izquierda. En este ejercicio es redundante, porque `MaterialApp` ya lo establece internamente; se deja de forma explícita porque en programas sin `MaterialApp` (solo con widgets básicos) sí es obligatorio, y sirve para entender que la dirección del texto es una dependencia real del layout.
6. `Padding(padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 100), ...)` Deja un margen invisible: **20** a izquierda y derecha, **100** arriba y abajo (`symmetric` aplica el mismo valor a ambos lados de cada eje). Las unidades son *píxeles lógicos* (dp), independientes de la densidad de la pantalla. El `const` significa que este objeto se crea una sola vez, en compilación.
7. `Center(child: ...)` Centra a su hijo en el espacio disponible. Le pasa restricciones **holgadas**: el hijo puede elegir su tamaño hasta el máximo disponible.
8. `SizeBox(height: 200, child: ...)` Fuerza que su hijo tenga **alto exactamente 200**. Como no se indica `width`, el ancho queda libre: el `Row` ocupará todo el ancho disponible.
9. `Row(children: [...])` Coloca sus hijos **en horizontal**. Su eje principal es horizontal (ocupa todo el ancho, `mainAxisSize.max`). Sus tres hijos, en orden de izquierda a derecha: rectángulo naranja, espacio, rectángulo azul.
10. `Expanded(flex: 4, child: Container(...))` Le dice a la fila: "este hijo se reparte el espacio libre, y le corresponden **4 partes**". El segundo `Expanded` tiene `flex: 1` (**1 parte**). Total: 5 partes. Por eso el naranja mide 4/5 y el azul 1/5 del espacio libre.
11. `Container(decoration: BoxDecoration(...))` Un `Container` sin `child` ni tamaño se **expande** hasta ocupar todo lo que le permitan las restricciones. Aquí el ancho lo impone `Expanded` (estricto) y el alto lo limita el `SizeBox` (200). Por eso cada rectángulo mide 200 de alto.
 - `color: Colors.orange / Colors.blue`: color de relleno. Está dentro de `BoxDecoration` (no directamente en `Container`) porque también se necesitan las esquinas redondeadas. **Un `Container` no admite `color`: y `decoration`: a la vez** (produce un error).
 - `borderRadius: BorderRadius.circular(20)`: esquinas redondeadas con radio 20.
 - `BorderRadius.circular` no es un constructor `const`, por eso esta parte del árbol no lleva `const`.
12. `const SizeBox(width: 20)` Un espaciador horizontal fijo de 20 entre los dos rectángulos. Sin `child`, un `SizeBox` es simplemente un hueco.

2.4 Cálculo de los tamaños (con un ejemplo)

Supón una pantalla de **360 dp** de ancho:

Paso	Cálculo	Resultado
Ancho útil de la fila	$360 - 20 - 20$ (padding lateral)	320
Espacio libre	$320 - 20$ (el <code>SizeBox</code> del medio)	300
Valor de 1 parte de flex	$300 \div (4 + 1)$	60
Ancho del naranja	4×60	240
Ancho del azul	1×60	60

Ambos miden **200 de alto**.

2.5 ¿Qué pasaría si...?

- **Quito el `SizeBox(height: 200)`**: el `Row` no tendría alto fijo y los rectángulos crecerían hasta llenar la altura disponible (la pantalla menos los márgenes).
- **Quito los `Expanded`**: un `Container` vacío dentro de un `Row` recibe ancho no acotado y, al no tener contenido, se reduce a **ancho 0**: los rectángulos desaparecerían.
- **Cambio `flex: 4` y `flex: 1` por `flex: 1` y `flex: 1`**: las dos cajas miden lo mismo.
- **Cambio `Row` por `Column`**: los rectángulos se apilan; el `flex` repartiría entonces el **alto**, no el ancho (y habría que dar un ancho al `SizeBox`).

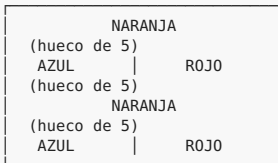
2.6 Ideas clave del Ejercicio 1

- `Row + Expanded(flex: n)` sirve para repartir el ancho en **proporciones**.
 - `SizeBox` es tanto una caja de tamaño fijo como un **espaciador**.
 - `Container` sin hijo se expande; con `decoration` se pueden dibujar formas de color con esquinas redondeadas.
 - Se lee el árbol de afuera hacia adentro: margen → centrado → alto fijo → fila → dos cajas.
-

3. Ejercicio 2: bandas con `Column` y `Row` anidados

Objetivo: llenar **toda la pantalla** con 4 bandas horizontales de igual altura, alternando una banda naranja completa con una banda dividida en azul (izquierda) y rojo (derecha), separadas por espacios de 5.

Resultado visual (esquema):



entre AZUL y ROJO también hay un hueco de 5

Los huecos de 5 no se pintan: dejan ver el fondo del `Scaffold` (blanco).

3.1 Código completo

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    MaterialApp(
      home: Scaffold(
        body: Directionality(
          textDirection: TextDirection.ltr,
          child: Column(
            children: [
              // NARANJA
              Expanded(
                child: Container(
                  color: Colors.orange,
                ),
              ),
              const SizedBox(height: 5),
              // AZUL + ROJO
              Expanded(
                child: Row(
                  children: [
                    Expanded(
                      child: Container(
                        color: Colors.blue,
                      ),
                    ),
                    const SizedBox(width: 5),
                    Expanded(
                      child: Container(
                        color: Colors.red,
                      ),
                    ),
                  ],
                ),
              ),
              // ESPACIO VERTICAL
              const SizedBox(height: 5),
              // NARANJA
              Expanded(
                child: Container(
                  color: Colors.orange,
                ),
              ),
              // ESPACIO VERTICAL
              const SizedBox(height: 5),
              // AZUL + ROJO
              Expanded(
                child: Row(
                  children: [
                    Expanded(
                      child: Container(
                        color: Colors.blue,
                      ),
                    ),
                    const SizedBox(width: 5),
                    Expanded(
                      child: Container(
                        color: Colors.red,
                      ),
                    ),
                  ],
                ),
              ),
            ],
          ),
        ),
      ),
    ),
  );
}
```

```

    ),
  ),
),
),
),
);
}

```

3.2 Árbol de widgets

```

MaterialApp
├── Scaffold
│   ├── Directionality (ltr)
│   │   └── Column
│   │       ├── Expanded → Container (naranja)
│   │       ├── SizedBox (alto 5)
│   │       ├── Expanded → Row
│   │       │   ├── Expanded → Container (azul)
│   │       │   ├── SizedBox (ancho 5)
│   │       │   └── Expanded → Container (rojo)
│   │       ├── SizedBox (alto 5)
│   │       ├── Expanded → Container (naranja)
│   │       ├── SizedBox (alto 5)
│   │       └── Expanded → Row
│   │           ├── Expanded → Container (azul)
│   │           ├── SizedBox (ancho 5)
│   │           └── Expanded → Container (rojo)

```

3.3 Explicación paso a paso

1. **Estructura exterior** (`import`, `main`, `runApp`, `MaterialApp`, `Scaffold`, `Directionality`): igual que en el Ejercicio 1 (ver sección 2.3, pasos 1 a 5). Aquí **no hay Padding ni Center**, así que el contenido ocupa toda la pantalla.
2. `Column(children: [...])` Coloca sus hijos **en vertical**, de arriba hacia abajo. Al ser hijo directo del `body`, su eje principal (vertical) ocupa toda la altura disponible.
3. **Los comentarios** `// NARANJA`, `// AZUL + ROJO`, `// ESPACIO VERTICAL`, `// ESPACIO HORIZONTAL` no hacen nada al ejecutar; solo documentan la intención de cada bloque. Es una buena práctica cuando el árbol es largo.
4. `Expanded(child: Container(color: Colors.orange))` (bandas naranjas) - Sin `flex`, el valor por defecto es **1**. - Como hijo de un `Column`, `Expanded` reparte **el alto** libre. - `Container(color: ...)` es la forma corta cuando solo se necesita un color de relleno (no hay bordes redondeados, así que no hace falta `BoxDecoration`). - Al no tener hijo, el `Container` se expande para llenar tanto el alto asignado como todo el ancho disponible.
5. `const SizedBox(height: 5)` Espaciador **vertical** de 5 entre bandas. Dentro de un `Column`, el `height` es el que tiene efecto (el `width` no).
6. `Expanded(child: Row(...))` (bandas azul + rojo) - El `Expanded` exterior (dentro del `Column`) le da a la fila **una banda de alto**. - El `Row` divide esa banda horizontalmente con dos `Expanded` internos, cada uno con `flex 1` (mitad y mitad), y un `SizedBox(width: 5)` en medio. - Ojo con los dos niveles de `Expanded`: el de fuera reparte el **alto** (el padre es `Column`); los de dentro reparten el **ancho** (el padre es `Row`). Cada `Expanded` actúa siempre sobre el eje de **su padre inmediato**.
7. **Orden final (arriba → abajo)**: naranja, azul|rojo, naranja, azul|rojo.

3.4 Cálculo de los tamaños

- **Alto**: hay 3 espaciadores de 5, es decir, 15 de espacio fijo. Los 4 `Expanded` (todos con `flex 1`) se reparten `(alto de pantalla - 15) ÷ 4`. Con una pantalla de 800 dp de alto: $(800 - 15) ÷ 4 = \mathbf{196,25}$ por banda.
- **Ancho de azul y rojo**: `(ancho - 5) ÷ 2`. Con 360 dp de ancho: $(360 - 5) ÷ 2 = \mathbf{177,5}$ cada uno.
- Las bandas naranjas ocupan todo el ancho (360 dp en este ejemplo).

3.5 Observaciones y mejoras

- **Sin SafeArea**: como no hay `AppBar` ni `SafeArea`, el contenido se dibuja incluso debajo de la barra de estado y de la muesca (`notch`) del celular. Para evitarlo se puede envolver el `Column` en un `SafeArea`.
- **Código repetido**: las filas azul|rojo están duplicadas. Se puede reutilizar con una función auxiliar:

```
Widget _filaAzulRoja() {
  return Expanded(
    child: Row(
      children: [
        Expanded(child: Container(color: Colors.blue)),
        const SizedBox(width: 5),
        Expanded(child: Container(color: Colors.red)),
      ],
    ),
  );
}
```

- **Espaciado más corto:** en versiones recientes de Flutter, `Row` y `Column` incluyen la propiedad `spacing` (por ejemplo `Column(spacing: 5, children: [...])`), que evita escribir cada `SizedBox` a mano. Conviene comprobar que la versión instalada la soporte.

3.6 Diferencias entre el Ejercicio 1 y el 2

	Ejercicio 1	Ejercicio 2
Widget principal	<code>Row</code>	<code>Column</code> con <code>Row</code> dentro
Uso de <code>flex</code>	Explícito (4 y 1)	Implícito (todos 1)
Tamaño del dibujo	Fijo en alto (200) y con márgenes	Ocupa toda la pantalla
Decoración	<code>BoxDecoration</code> con esquinas redondeadas	<code>Container(color: ...)</code> simple
Concepto nuevo	Proporciones con <code>flex</code>	Anidar filas dentro de columnas

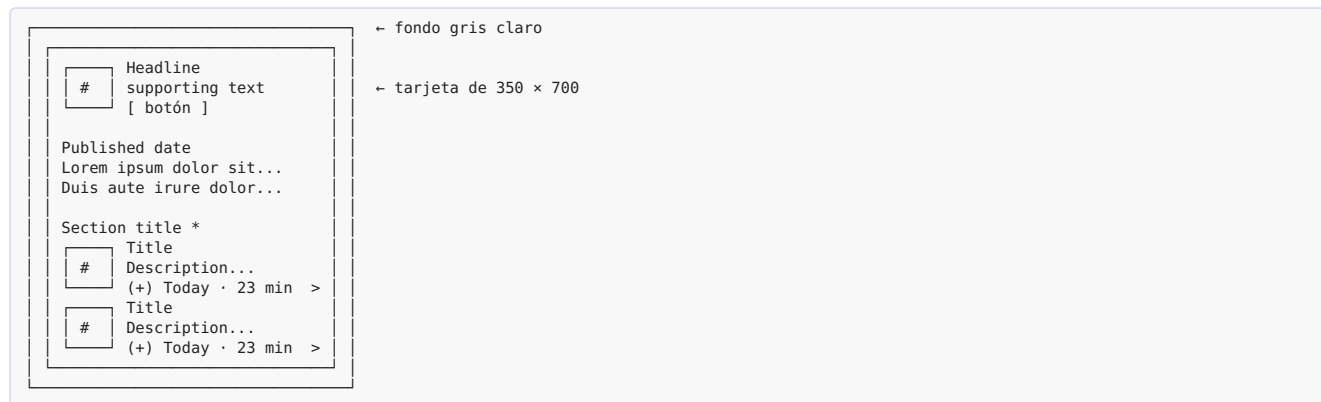
3.7 Ideas clave del Ejercicio 2

- Se pueden **anidar** `Row` y `Column` para construir cuadrículas y estructuras complejas.
- Cada `Expanded` reparte el espacio del eje de su **padre directo**.
- `SizedBox` con `width` o `height` funciona como separador entre elementos.
- Un `Container(color: ...)` sin hijo es la manera más rápida de dibujar un bloque de color.

4. Ejercicio 3: pantalla tipo artículo (estilo Material 3)

Objetivo: reproducir el diseño de una tarjeta de contenido: una cabecera con miniatura, título, subtítulo y un botón decorativo; un bloque de texto con fecha y dos párrafos; un título de sección con ícono; y una lista de dos elementos multimedia. Es el primer ejercicio con **clases propias** (`MyApp`) y con **métodos auxiliares** para reutilizar partes de la interfaz.

Resultado visual (esquema):



4.1 Código completo

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        backgroundColor: const Color(0xffdcddc),
        body: Center(
          child: SizedBox(
            height: 700,
            width: 350,
            child: Container(
              padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 20),
              decoration: BoxDecoration(
                color: const Color(0xfffbf8fd),
                borderRadius: BorderRadius.circular(24),
              ),
            ),
          child: SingleChildScrollView(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                // Fila superior: Miniatura + Encabezado + Botón
                Row(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    _buildThumbnail(size: 90),
                    const SizedBox(width: 16),
                    Expanded(
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          const Text(
                            'Headline',
                            style: TextStyle(
                              fontSize: 22,
                              fontWeight: FontWeight.bold,
                              color: Colors.black87,
                            ),
                          ),
                          const SizedBox(height: 4),
                          const Text(
                            'supporting text',
                            style: TextStyle(
                              fontSize: 14,
                              color: Colors.black54,
                            ),
                          ),
                          const SizedBox(height: 12),
                          Container(
                            width: 75,
                            height: 32,
                            decoration: BoxDecoration(
```



```

}

static Widget _buildListItem() {
  return Row(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      _buildThumbnail(size: 80),
      const SizedBox(width: 14),
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text(
              'Title',
              style: TextStyle(
                fontSize: 16,
                fontWeight: FontWeight.bold,
                color: Colors.black87,
              ),
            ),
            const SizedBox(height: 3),
            const Text(
              'Description duis aute irure dolor in reprehenderit in voluptate velit.',
              maxLines: 2,
              overflow: TextOverflow.ellipsis,
              style: TextStyle(
                fontSize: 12,
                color: Colors.black54,
                height: 1.25,
              ),
            ),
            const SizedBox(height: 8),
            const Row(
              children: [
                Icon(
                  Icons.add_circle_outline,
                  size: 17,
                  color: Colors.black54,
                ),
                SizedBox(width: 5),
                Text(
                  'Today · 23 min',
                  style: TextStyle(
                    fontSize: 11,
                    color: Colors.black54,
                  ),
                ),
                Spacer(),
                Icon(
                  Icons.play_arrow,
                  size: 20,
                  color: Colors.black87,
                ),
              ],
            ),
          ],
        ),
      ),
    ],
  );
}

```

4.2 Árbol de widgets

Pantalla principal (MyApp.build):

```

MyApp (StatelessWidget)
├─ MaterialApp (sin banner de debug)
│   └─ Scaffold (fondo gris #DCDCDC)
│       └─ Center
│           └─ SizedBox 350 × 700
│               └─ Container (tarjeta: fondo lila muy claro, esquinas 24, padding 16/20)
│                   └─ SingleChildScrollView
│                       └─ Column (alineada a la izquierda)
│                           └─ Row ← cabecera
│                               └─ _buildThumbnail(90)
│                               └─ SizedBox (ancho 16)
│                               └─ Expanded
│                                   └─ Column: "Headline", "supporting text", botón decorativo
│                                       └─ SizedBox (18)
│                                       └─ Text "Published date"
│                                       └─ SizedBox (8)
│                                       └─ Text (párrafo 1)
│                                       └─ SizedBox (12)
│                                       └─ Text (párrafo 2)
│                                       └─ SizedBox (20)
│                                       └─ Row: "Section title" + SizedBox(8) + Icon estrella
│                                       └─ SizedBox (14)
│                                       └─ _buildListItem()
│                                       └─ SizedBox (12)
│                                       └─ _buildListItem()

```

Cada elemento de la lista (`_buildListItem`):

```
Row (alineada arriba)
├─ _buildThumbnail(80)
├─ SizedBox (ancho 14)
└─ Expanded
    └─ Column (alineada a la izquierda)
        ├── Text "Title"
        ├── SizedBox (3)
        ├── Text "Description..." (máx. 2 líneas, con "...")
        ├── SizedBox (8)
        └─ Row: Icon (+) · SizedBox(5) · Text "Today · 23 min" · Spacer · Icon ▶
```

La miniatura (`_buildThumbnail`):

```
Container (size × size, color lila grisáceo, esquinas 16)
└─ Center
    └─ Icon (interests_rounded, tamaño 45)
```

4.3 Explicación bloque por bloque

A. La clase `MyApp`

1. `void main() { runApp(const MyApp()); }` Ahora `runApp` recibe una clase propia. `const` porque `MyApp` tiene constructor constante.
2. `class MyApp extends StatelessWidget` Define un widget **sin estado**: no guarda datos que cambien; siempre dibuja lo mismo.
3. `const MyApp({super.key});` Constructor constante. `super.key` pasa la `key` opcional al constructor de `StatelessWidget`. Las `keys` ayudan a Flutter a identificar widgets cuando cambian de posición; aquí no se usan, pero es la convención escribirlo.
4. `@override Widget build(BuildContext context)` Reemplaza el método `build` heredado: aquí se define **qué dibuja** el widget. Devuelve el árbol descrito arriba.

B. `MaterialApp` y `Scaffold`

5. `debugShowCheckedModeBanner: false` Oculta la etiqueta roja "DEBUG" que aparece en la esquina superior derecha mientras se ejecuta en modo *debug*.
6. `backgroundColor: const Color(0xffdcddcdc)` Color de fondo de la pantalla. **Cómo leer un color hexadecimal en Flutter:** `0xAARRGGBB` → `AA` = opacidad (`ff` = 100 % opaco), luego rojo, verde y azul. `0xffdcddcdc` es un gris claro (`R = G = B = dc = 220`).

C. La tarjeta central

7. `Center` → `SizedBox(height: 700, width: 350)` Centra una caja de **350 × 700**. Probablemente se eligieron estas medidas para simular una pantalla de celular cuando se ejecuta en un monitor ancho (web o escritorio). Si la pantalla real es más pequeña, el `SizedBox` se ajusta a las restricciones de su padre, así que no se sale de la pantalla.
8. `Container(padding: ..., decoration: ...)` La tarjeta. - `padding: EdgeInsets.symmetric(horizontal: 16, vertical: 20)`: espacio interior de 16 a los lados y 20 arriba/abajo. (Aquí `Container` incluye el relleno sin necesitar un `Padding` aparte.) - `color: Color(0xffffbf8fd)`: blanco con un tinte lila casi imperceptible. - `borderRadius: BorderRadius.circular(24)`: esquinas muy redondeadas. - Ancho útil dentro de la tarjeta: $350 - 16 - 16 = 318$.
9. `SingleChildScrollView` Envuelve el contenido para que, si es más alto que la tarjeta, se pueda **hacer scroll** en lugar de mostrar el error de desbordamiento (*overflow*, las franjas amarillas y negras). Dentro de un scroll vertical, el `Column` recibe altura ilimitada y ocupa solo lo que necesita su contenido.
10. `Column(crossAxisAlignment: CrossAxisAlignment.start)` Apila todo el contenido en vertical. Como el eje cruzado de un `Column` es el horizontal, `start` alinea los hijos **a la izquierda**. Sin esto, por defecto (`center`) los textos quedarían centrados.

D. La cabecera (primera `Row`)

11. `Row(crossAxisAlignment: CrossAxisAlignment.start)` Fila horizontal. Aquí el eje cruzado es el **vertical**, así que `start` alinea los elementos **arriba** (por defecto se centrarían verticalmente).
12. `_buildThumbnail(size: 90)` Llama al método auxiliar (ver sección F) para dibujar un cuadro de 90 × 90.
13. `const SizedBox(width: 16)` Separación horizontal entre la miniatura y el texto.
14. `Expanded(child: Column(...))` Punto clave. Dentro de un `Row`, un hijo **no flexible** recibe ancho ilimitado; un `Column` con textos largos intentaría ser más ancho que la pantalla y se produciría un error de desbordamiento. Con `Expanded`, el `Column` recibe exactamente el **ancho restante**: $318 - 90 - 16 = 212$, y así los textos pueden partirse en varias líneas.

15. Los tres elementos del `Column` de texto:

- `Text('Headline', ...)`: el título principal.
- `Text('supporting text', ...)`: texto de apoyo, más pequeño y más claro.
- `Container(width: 75, height: 32, decoration: ...)`: un rectángulo de **75 × 32** color `0xff6750a4` (el morado base de Material 3) con radio 16. Como el radio es la mitad del alto, queda con forma de **píldora**; simula un botón, pero no es interactivo (no tiene `onTap`, ni texto).

E. Textos: fecha, párrafos y título de sección

16. `const SizedBox(height: 18), 8, 12, 20, 14` Separadores verticales. Fijan el "ritmo" del diseño.

17. `Text('Published date', ...)` Texto pequeño (11), en negrita y color atenuado.

18. Los dos párrafos "**Lorem ipsum**" *Lorem ipsum* es texto de relleno tradicional en diseño. Detalles de código:

- Los textos largos se parten en varias cadenas consecutivas entre comillas simples: Dart las **concatena** automáticamente en una sola.
- `height: 1.35` es un **multiplicador del interlineado**: la altura de línea es $1,35 \times \text{fontSize}$ ($12 \times 1,35 = 16,2$).

19. `const Row(children: [Text, SizedBox, Icon])` Título de sección con un ícono a su derecha. Como todo lo de adentro es constante, la `Row` puede llevar `const` y los hijos ya no repiten `const`. `Icons.stars_rounded` es un ícono de la fuente Material Icons.

20. `_buildListItem()` **dos veces** Se reutiliza el mismo método para no duplicar código, con un `SizedBox(height: 12)` entre ambos.

F. Los métodos auxiliares

21. `static Widget _buildThumbnail({double size = 80})`

- `static`: pertenece a la clase, no necesita una instancia.
- `_`: privado al archivo.
- `{double size = 80}`: parámetro **nombrado opcional** con valor por defecto 80. Por eso se puede llamar `_buildThumbnail(size: 90)` o `_buildThumbnail()`.
- Devuelve un `Container` cuadrado (`width` y `height` iguales a `size`) con color `0xffebe5ed` y radio 16.
- `child: const Center(child: Icon(Icons.interests_rounded, ...))`: un ícono centrado que hace de imagen de relleno (*placeholder*). Color `0xffcac4d0`, tamaño 45.

22. `static Widget _buildListItem()` Devuelve una fila de elemento de lista:

- Miniatura de 80 + espacio de 14 + `Expanded` con la columna de texto (ancho: $318 - 80 - 14 = 224$).
- `Text('Title', ...)`: 16, negrita.
- `Text('Description...', maxLines: 2, overflow: TextOverflow.ellipsis, ...)`: **limita el texto a 2 líneas** y, si es más largo, lo corta con puntos suspensivos ("...").
- Fila inferior: ícono `add_circle_outline` (17) + espacio (5) + texto `Today · 23 min` (11) + `Spacer()` + ícono `play_arrow` (20).
- `Spacer()` es un espacio flexible que ocupa todo el espacio libre de la fila: empuja el ícono de "play" hasta el extremo derecho.

4.4 Tabla de estilos de texto

`Colors.black87` es negro con 87 % de opacidad (texto principal) y `Colors.black54` es negro con 54 % (texto secundario). Así se logra una **jerarquía visual** sin usar otros colores.

Texto	Tamaño	Peso	Color
Headline	22	bold	black87
supporting text	14	normal	black54
Published date	11	bold	black54
Párrafos	12 (height 1,35)	normal	black87
Section title	19	bold	black87
Title (lista)	16	bold	black87
Description (lista)	12 (height 1,25)	normal	black54
Today · 23 min	11	normal	black54

4.5 Ritmo vertical (espaciadores)

cabecera → 18 → fecha → 8 → párrafo 1 → 12 → párrafo 2 → 20 → título de sección → 14 → elemento → 12 → elemento

Dentro de la cabecera: `Headline` → 4 → `supporting text` → 12 → botón. Dentro de cada elemento de la lista: `Title` → 3 → descripción → 8 → fila inferior.

4.6 Ideas clave del Ejercicio 3

- Un diseño complejo se descompone en **filas y columnas anidadas** más espaciadores.
 - **Expanded dentro de un Row es necesario** cuando el hijo contiene texto que debe ajustarse al ancho restante.
 - `crossAxisAlignment` cambia la alineación en el eje perpendicular al principal: en `Column` (horizontal) y en `Row` (vertical).
 - Se pueden **extraer métodos auxiliares** (`_buildThumbnail`, `_buildListItem`) para reutilizar y ordenar el código. Nota: en Flutter suele recomendarse extraer a **clases** `StatelessWidget` en vez de métodos (así lo hace `Resumen.dart` con `_MediaItemRow`), porque Flutter puede optimizarlas mejor.
 - `SingleChildScrollView` evita errores de desbordamiento cuando el contenido es más alto que el espacio.
 - `maxLines` + `overflow: TextOverflow.ellipsis` recortan textos largos.
 - `Spacer` empuja elementos a los extremos de una fila o columna.
-

5. EjercicioWidgets.dart : estado y setState (resumen)

Qué hace: muestra una cuadrícula de 2 × 2 cuadros de colores. Cada cuadro tiene un `flex` distinto, así que sus anchos son proporcionales. Un botón "Rotar valores" desplaza cíclicamente los colores y las proporciones, y la pantalla se redibuja.

Concepto principal: `StatefulWidget` y `setState`.

5.1 Estructura de un `StatefulWidget` (dos clases)

```
class EjercicioRotacionPantalla extends StatefulWidget {
  const EjercicioRotacionPantalla({super.key});

  @override
  State<EjercicioRotacionPantalla> createState() =>
    _EjercicioRotacionPantallaState();
}

class _EjercicioRotacionPantallaState extends State<EjercicioRotacionPantalla> {
  // aquí viven las variables que cambian y el método build
}
```

- La primera clase (el widget) es **immutable**; su único trabajo es crear el objeto `State`.
- La segunda clase (el `State`) guarda los datos que cambian y define `build`.
- El nombre del `State` empieza con `_` (privado).

5.2 El estado y la rotación

```
final List<Color> _colors = [
  Colors.amber, Colors.blue, Colors.deepOrange, Colors.green
];
final List<int> _proportions = [1, 2, 3, 4];

void _rotarValores() {
  setState(() {
    _colors.insert(0, _colors.removeLast());
    _proportions.insert(0, _proportions.removeLast());
  });
}
```

- `removeLast()` **quita y devuelve** el último elemento; `insert(0, x)` lo coloca **al inicio**. El efecto es una rotación circular hacia la derecha.
- `setState(() { ... })` ejecuta los cambios y le avisa a Flutter que el estado cambió, para que vuelva a llamar a `build` y redibuje. Si se modifican las listas **sin** `setState`, los datos cambian pero la pantalla no.
- Las listas son `final` pero mutables: `final` impide reasignar la variable, no modificar su contenido.

Evolución de los datos con cada pulsación:

Pulsación	colors	proportions
Inicial	amber, blue, deepOrange, green	1, 2, 3, 4
1	green, amber, blue, deepOrange	4, 1, 2, 3
2	deepOrange, green, amber, blue	3, 4, 1, 2
3	blue, deepOrange, green, amber	2, 3, 4, 1
4	amber, blue, deepOrange, green	1, 2, 3, 4 (vuelve al inicio)

5.3 El layout

- `Scaffold` (fondo blanco) → `SafeArea` (evita la barra de estado y la muesca) → `Padding` horizontal 16 → `Column`(`mainAxisAlignment: MainAxisAlignment.center`) (centra todo verticalmente).
- Un `SizeBox`(`height: 350`) contiene un `Column` con dos filas `Expanded`, separadas por un `SizeBox`(`height: 8`).
- Cada fila tiene dos cuadros creados con el método `_crearCuadro(color, flex)`, separados por `SizeBox`(`width: 8`). Fila 1: posiciones 0 y 1; fila 2: posiciones 2 y 3.
- Después, `SizeBox`(`height: 30`) y el botón.

```
Widget _crearCuadro(Color color, int flexValue) {
  return Expanded(
    flex: flexValue,
    child: Container(
      decoration: BoxDecoration(
        color: color,
        borderRadius: BorderRadius.circular(8.0),
      ),
    ),
  );
}
```

Como el `flex` viene de `_proportions`, al rotar los valores **cambian los anchos** de los cuadros: en la posición 0 el `flex` pasa de 1 a 4, luego 3, etc.

5.4 El botón

```
ElevatedButton(
  onPressed: _rotarValores,
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.purple.shade50,
    foregroundColor: Colors.deepPurple,
    elevation: 0,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(20)),
  ),
  child: const Text('Rotar valores'),
)
```

- `onPressed: _rotarValores` **pasa la función** (sin paréntesis) para que Flutter la ejecute al presionar. Si se escribiera `_rotarValores()` se ejecutaría al construir el widget.
- `styleFrom` configura: `backgroundColor` (fondo), `foregroundColor` (color del texto), `elevation: 0` (sin sombra) y `shape` (forma con esquinas redondeadas).

5.5 Nota sobre los comentarios

Algunos comentarios del archivo terminan con marcas como `[cite: 1]` o `[cite: 2]`. No afectan la ejecución, pero parecen residuo de haber copiado texto de otra fuente, así que conviene borrarlas para dejar el código limpio.

5.6 Ideas clave

- **StatefulWidget = widget + State**; los datos que cambian van en el `State`.
- **Nada se redibuja sin `setState`**.
- El `flex` no tiene que ser una constante: puede venir de una variable y cambiar con el tiempo.
- Pasar una función (`onPressed: f`) no es lo mismo que llamarla (`f()`).

6. Resumen.dart : InheritedWidget , pestañas y más (resumen)

Qué hace: una app con una barra superior y **dos pestañas** que reúnen los ejercicios anteriores:

1. **"Rotación Flex"**: la cuadrícula rotativa, ahora con `GestureDetector` (tocar un cuadro también rota).
2. **"Layout Sem 3"**: la pantalla tipo artículo del Ejercicio 3, pero con `ListView` y una clase `MediaItemRow`.

El título y el color de la barra superior provienen de un `InheritedWidget` propio. El archivo está dividido en secciones numeradas con comentarios.

6.1 Sección 1: InheritedWidget (AppConfigInherited)

Un `InheritedWidget` permite que **cualquier widget descendiente** lea datos que están más arriba en el árbol, sin pasarlos de constructor en constructor. Es el mecanismo detrás de `Theme.of(context)` y `MediaQuery.of(context)`.

```
class AppConfigInherited extends InheritedWidget {
  final String appTitle;
  final Color primaryColor;

  const AppConfigInherited({
    super.key,
    required this.appTitle,
    required this.primaryColor,
    required super.child,
  });

  static AppConfigInherited? of(BuildContext context) {
    return context.dependOnInheritedWidgetOfExactType<AppConfigInherited>();
  }

  @override
  bool updateShouldNotify(AppConfigInherited oldWidget) {
    return appTitle != oldWidget.appTitle ||
           primaryColor != oldWidget.primaryColor;
  }
}
```

- **Campos final**: los datos compartidos (título y color).
- **required super.child**: el subárbol que podrá leer esos datos.
- **static ... of(context)**: método de conveniencia. `dependOnInheritedWidgetOfExactType<T>()` **sube por el árbol** hasta encontrar el `T` más cercano y registra una dependencia. Devuelve `AppConfigInherited?` (puede ser `null` si no hay ninguno arriba).
- **updateShouldNotify**: Flutter lo llama cuando el `InheritedWidget` se reconstruye; devuelve `true` si los datos cambiaron, y entonces se redibujan los widgets dependientes.

6.2 Sección 2: widget raíz (MyApp)

`MyApp` envuelve al `MaterialApp` dentro de `AppConfigInherited(appTitle: 'Programación Móvil - Semana 3', primaryColor: Color(0xFF6750A4))`. Así **todas las pantallas** de la app quedan por debajo y pueden leer la configuración.

6.3 Sección 3: MainTabScreen (StatelessWidget)

```
final config = AppConfigInherited.of(context);
```

Lee la configuración. Como `of` devuelve un valor que puede ser nulo, se usa `config?.appTitle ?? 'Widgets Demo'` (si no hay configuración, se usa un título por defecto).

Estructura de las pestañas:

- **DefaultTabController(length: 2, ...)**: crea y comparte el controlador de pestañas. `length` debe coincidir con el número de pestañas.
- **AppBar** con `title`, `backgroundColor` y `foregroundColor` (color de textos e íconos), y `bottom: TabBar(...)` con dos `Tab` (cada uno con ícono y texto).
- **TabBarView(children: [...])**: el contenido de cada pestaña. **El orden de los hijos debe coincidir con el de los Tab**.

6.4 Sección 4: RotatingFlexExercise (StatefulWidget)

Es la versión ampliada de `EjercicioWidgets.dart`:

- Mismas listas `_colors` y `_proportions`, mismo `_rotateValues` con `setState`, aunque escrito en pasos más explícitos (`final lastColor = _colors.removeLast(); _colors.insert(0, lastColor);`).
- **Orden visual distinto**: la fila superior usa las posiciones `[3]` y `[0]` y la inferior las posiciones `[1]` y `[2]`.

- `Spacer(flex: 1)` **arriba** y `Spacer(flex: 2)` **abajo**: reparten el espacio libre vertical en proporción 1 : 2, de modo que la cuadrícula queda un poco por encima del centro.
- Cada cuadro es un `_ColoredCard` con una etiqueta `'Flex: ${_proportions[i]}'`, y **tocar cualquier cuadro también rota** los valores.

6.5 Widget auxiliar `_ColoredCard`

```
class _ColoredCard extends StatelessWidget {
  final Color color;
  final String label;
  final VoidCallback onTap;

  const _ColoredCard({
    required this.color,
    required this.label,
    required this.onTap,
  });
  // build: GestureDetector(onTap: onTap, child: Container(... Center(Text(label))))
}
```

- `VoidCallback` es el tipo de una función sin parámetros que no devuelve nada (`void Function()`).
- `GestureDetector` detecta gestos (toque, doble toque, arrastre...) sobre su hijo; `onTap` se ejecuta al tocar.
- El widget es `StatelessWidget`: no guarda estado, solo recibe datos y una función desde su padre. La rotación la hace el padre con `setState`. Este patrón, "los datos bajan y los eventos suben", es muy común en Flutter.
- `required this.x` declara parámetros nombrados obligatorios y los asigna directamente a los campos.

6.6 Sección 5: `ArticleLayoutExercise` y `_MediaItemRow`

Es el mismo diseño del Ejercicio 3, con estas diferencias:

Ejercicio 3		<code>Resumen.dart</code> (pestaña 2)
Contenedor	<code>SizeBox 350×700 + SingleChildScrollView + Column</code>	<code>ListView</code> (ya tiene scroll y padding)
Elementos de lista	Método <code>_buildListItem()</code>	Clase <code>_MediaItemRow</code> (<code>StatelessWidget</code>)
Colores de texto	<code>Colors.black87</code> / <code>black54</code>	<code>Colors.grey.shade600</code> , <code>shade700</code>
Íconos	<code>interests_rounded</code> , <code>stars_rounded</code>	<code>widgets_outlined</code> , <code>auto_awesome</code> , <code>stars</code> (morado)

`ListView(padding: ..., children: [...])` es la forma más simple de una lista con scroll: no hace falta `SingleChildScrollView`.

6.7 Ideas clave de `Resumen.dart`

- `InheritedWidget`: compartir datos hacia abajo en el árbol sin pasarlos manualmente.
- `DefaultTabController` + `TabBar` + `TabBarView`: navegación por pestañas.
- `GestureDetector`: hacer interactivo cualquier widget.
- **Extraer widgets en clases** (`_ColoredCard`, `_MediaItemRow`) mejora la legibilidad y reutilización.
- `ListView` = lista con scroll integrado.

7. Chuleta final, errores típicos y autoevaluación

7.1 Mapa mental de layout

```
¿Quiero poner cosas una al lado de otra? → Row
¿Quiero apilarlas de arriba a abajo? → Column
¿Quiero que un hijo ocupe el espacio sobrante? → Expanded (con flex si hay varios)
¿Quiero un hueco fijo? → SizedBox(width/height)
¿Quiero un hueco que empuje a los demás? → Spacer
¿Quiero un margen alrededor? → Padding (o padding de Container)
¿Quiero color, bordes redondeados, tamaño? → Container + BoxDecoration
¿El contenido puede ser más alto que la pantalla? → SingleChildScrollView o ListView
¿Necesito que la pantalla cambie con la interacción? → StatefulWidget + setState
```

7.2 Errores típicos

Error / síntoma	Causa habitual	Solución
Franjas amarillas y negras ("RenderFlex overflowed")	El contenido es más grande que el espacio	Expanded, Flexible, SingleChildScrollView, o reducir tamaños
Incorrect use of ParentDataWidget	Un Expanded que no es hijo directo de Row, Column o Flex	Colocar el Expanded directamente dentro de la fila o columna
"Vertical viewport was given unbounded height"	Un ListView dentro de un Column sin restricción de alto	Envolver el ListView en Expanded o darle un alto fijo
Error por usar color y decoration en un Container	No se permiten a la vez	Poner el color dentro de BoxDecoration
El cuadro de color no se ve	Container vacío dentro de un Row sin Expanded ni tamaño (ancho 0)	Envolverlo en Expanded o darle width / height
La pantalla no se actualiza al cambiar una variable	Se modificó el dato sin setState	Hacer el cambio dentro de setState(() { ... })
Error de const ("not a constant expression")	Un const con algún valor no constante adentro (por ejemplo BorderRadius.circular)	Quitar el const de ese nivel o del padre
El contenido queda debajo de la barra de estado	No se usó SafeArea	Envolver el contenido en SafeArea

7.3 Autoevaluación

- En el Ejercicio 1, ¿por qué el naranja mide 4 veces más que el azul?** Porque tiene `flex: 4` frente a `flex: 1`; el espacio libre se reparte proporcionalmente (4/5 y 1/5).
- ¿Para qué sirve el `SizedBox(width: 20)` que está entre los dos `Expanded`?** Es un espacio fijo de 20 que se descuenta antes de repartir el resto con `flex`.
- En el Ejercicio 2, ¿por qué hay `Expanded` dentro de un `Expanded`?** El exterior reparte el alto dentro del `Column`; los interiores reparten el ancho dentro del `Row`. Cada `Expanded` actúa sobre el eje de su padre directo.
- En el Ejercicio 3, ¿qué ocurriría si se quita el `Expanded` que envuelve al `Column` de texto de la cabecera?** El `Column` recibiría ancho ilimitado dentro del `Row` y los textos largos provocarían un desbordamiento en lugar de partirse en líneas.
- ¿Qué diferencia hay entre `crossAxisAlignment: start` en un `Row` y en un `Column`?** En un `Row` alinea los hijos **arriba** (eje cruzado vertical); en un `Column`, **a la izquierda** (eje cruzado horizontal).
- ¿Por qué `onPressed: _rotarValores` no lleva paréntesis?** Porque se pasa la función como valor para que se ejecute al presionar; con paréntesis se ejecutaría de inmediato al construir el widget.
- ¿Qué hace `updateShouldNotify` en un `InheritedWidget`?** Indica si los widgets dependientes deben redibujarse cuando el `InheritedWidget` se reconstruye con datos distintos.
- ¿Cuándo conviene usar `StatelessWidget` y cuándo `StatefulWidget`?** `StatelessWidget` cuando lo que se muestra depende solo de los datos recibidos; `StatefulWidget` cuando el widget maneja datos propios que cambian con el tiempo o con la interacción.

7.4 Ejercicios de práctica sugeridos

- Modificar el Ejercicio 1 para tener **tres** cajas con proporción 2 : 1 : 1 y luego pasarlo a `Column`.
- Reescribir el Ejercicio 2 con la función auxiliar `_filaAzulRoja()` y con `SafeArea`.

- En el Ejercicio 3, convertir `_buildListItem` en una clase `StatelessWidget` que reciba `title`, `description` y `time` como parámetros y generar la lista con `List.generate` o `ListView.builder`.
- En `EjercicioWidgets.dart`, agregar un segundo botón que rote en el **sentido contrario**.
- Hacer que el botón decorativo del Ejercicio 3 sea real con `ElevatedButton` o `FilledButton`.